Lab 2 Report: Introduction to Embedded System Programming

Gagan Adhikari

December 6, 2025

Authorship Statement

I, Gagan Adhikari, declare that this lab report is the result of my own original work. Any outside sources used have been properly cited and acknowledged. This report has not been submitted previously for any academic or professional purpose.

I implemented the C code and logic manually. AI tools were used to refine latex code, but the design and debugging were performed by me.

Contents

1 Introduction **4**

2 Materials and Methods **4**
2.1 Materials 4
2.2 Methodology 4

3 Results **4**
3.1 Hardware Design 4
3.2 Software Implementation 6
3.3 Pseudocode 7
3.4 Test Cases and Observations 7
3.5 Discussion 8

4 References **9**

1 Introduction

This lab was part of the DT124A – Embedded Systems course at HKR. The objective was to design and implement a simple embedded program that monitors car status information using an 8-bit variable and displays results on LEDs. The lab introduced bitwise operations, condition checking, and hardware interfacing with the Atmega328p microcontroller.

2 Materials and Methods

2.1 Materials

- Atmega328p microcontroller
- Breadboard and jumper wires
- 8 LEDs with resistors
- Microchip Studio IDE for programming and debugging

2.2 Methodology

The status variable `stat` was defined as an 8-bit integer, with each bit representing a subsystem condition (brake, fluid, oil, temperature, etc.).

- The lowest four bits were mapped to LEDs to display raw status values.
- Additional LEDs were used to indicate logical conditions (all systems normal, engine too hot, oil low AND hot, brake issues).
- The program was written in C, compiled, and debugged using Microchip Studio, then uploaded to the Atmega328p.

3 Results

3.1 Hardware Design

The hardware consisted of LEDs connected to PORTD pins 0–7.

- Pins 0–3: Status LEDs (raw bits).
- Pins 4–7: Condition LEDs.

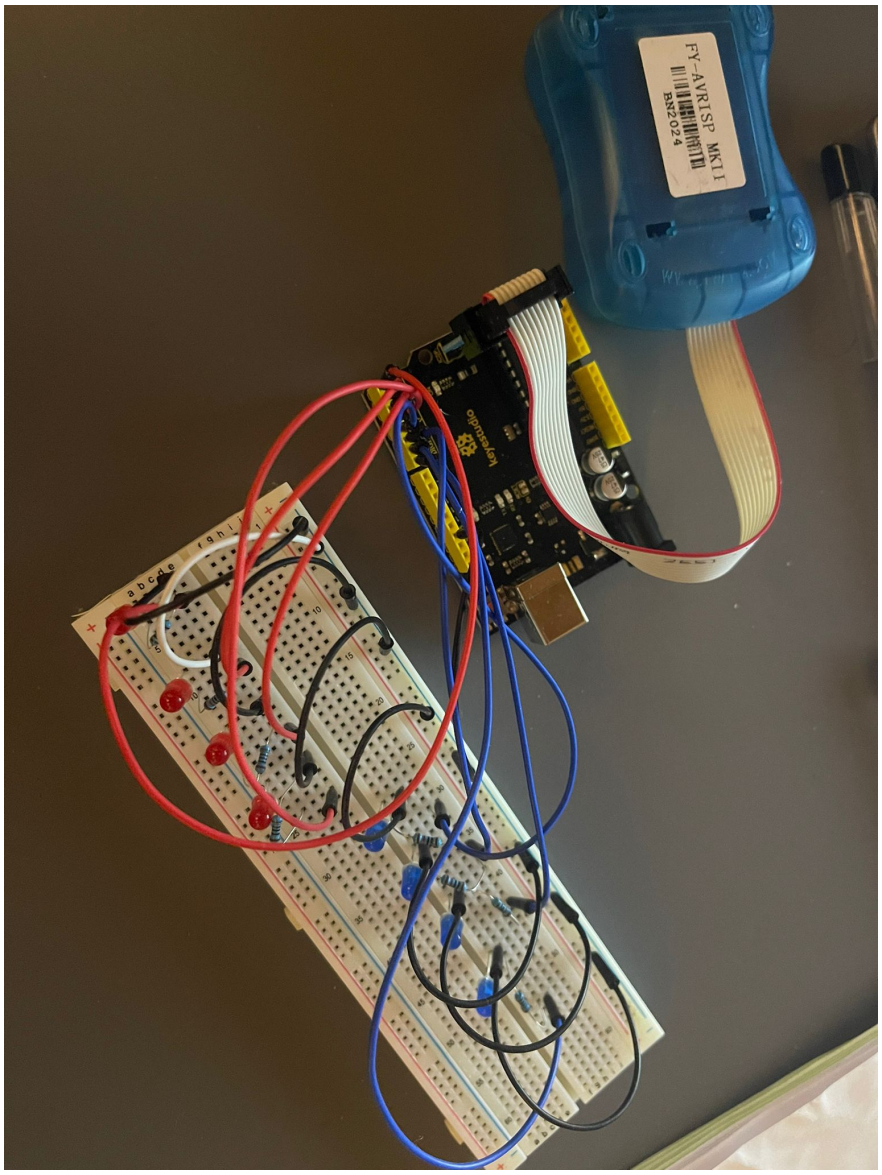


Figure 1: Hardware wiring diagram for Lab 2.

3.2 Software Implementation

The following C program was implemented: DDRD = 0xFF sets pins 0-7 as an output and stat 0x0F is to get stat only i.e 0-3 first least bits.

```
#include <avr/io.h>

void display_statuses_and_results(uint8_t stat);

int main(void)
{
    DDRD = 0xFF;  // Set PORTD pins 0{7 as output
    uint8_t stat = 0b01101101;  // Test case 1
    //uint8_t stat = 0B01010111; // test case 2
    //uint8_t stat = 0B00000000; // test case 3
    while (1)
    {
        display_statuses_and_results(stat);
    }
}

void display_statuses_and_results(uint8_t stat)
{
    uint8_t output = 0x00;

    // Status LEDs (bits 0{3)
    output |= (stat & 0x0F);

    // Condition LEDs
    if (stat == 0) output |= (1 << 4);  // All systems normal
    if (stat & (1 << 2)) output |= (1 << 5);  // Engine too hot
    if ((stat & (1 << 5)) && (stat & (1 << 6))) output |= (1 << 6);  // Oil low AND hot
    if ((stat & (1)) || (stat & (1 << 1))) output |= (1 << 7);  // Brake fluid OR pads b

    PORTD = output;
}
```

This AVR program uses an 8-bit status value and uses it to control LEDs connected to PORTD. The lower four bits of the status byte (bits 0–3) are directly copied to PORTD to show basic system statuses. Additional condition checks determine which other LEDs should turn on: if all status bits are zero, bit 4 is set to indicate that all systems are normal; if bit 2 is set, bit 5 lights up to show the engine is too hot; if both bits 5 and 6 are set in the input status, bit 6 is activated to indicate that the oil is both low and overheating; and if either bit 0 or bit 1 is set, bit 7 is turned on to warn about brake fluid or brake pad issues. All these outputs are combined into one byte and written to PORTD in a infinite loop.

3.3 Pseudocode

To simplify understanding, the program logic can be expressed as pseudocode:

```
FUNCTION display_statuses_and_results(stat):
    output = 0

    // Show raw status bits (0{3)
    output = output OR (stat AND 0x0F)

    // Condition checks
    IF stat == 0:
        SET LED4 (all systems normal)
    IF bit2 of stat == 1:
        SET LED5 (engine too hot)
    IF bit5 AND bit6 of stat == 1:
        SET LED6 (oil low AND hot)
    IF bit0 OR bit1 of stat == 1:
        SET LED7 (brake fluid OR pads bad)

    PORTD = output
END FUNCTION

MAIN:
    Configure PORTD as output
    stat = test case value
    WHIE LOOP WHEN TRUE:
        CALL display_statuses_and_results(stat)
```

3.4 Test Cases and Observations

Three test cases were used: 0-3 bits are for status.

Table 1: Test cases and observed LED outputs

Test Case	`stat` Value (Binary)	Observed LED Outputs
1	01101101	condition LEDs indicate brake issue + engine hot
2	01010111	condition LEDs indicate oil low + brake issue
3	00000000	All systems normal LED lights up

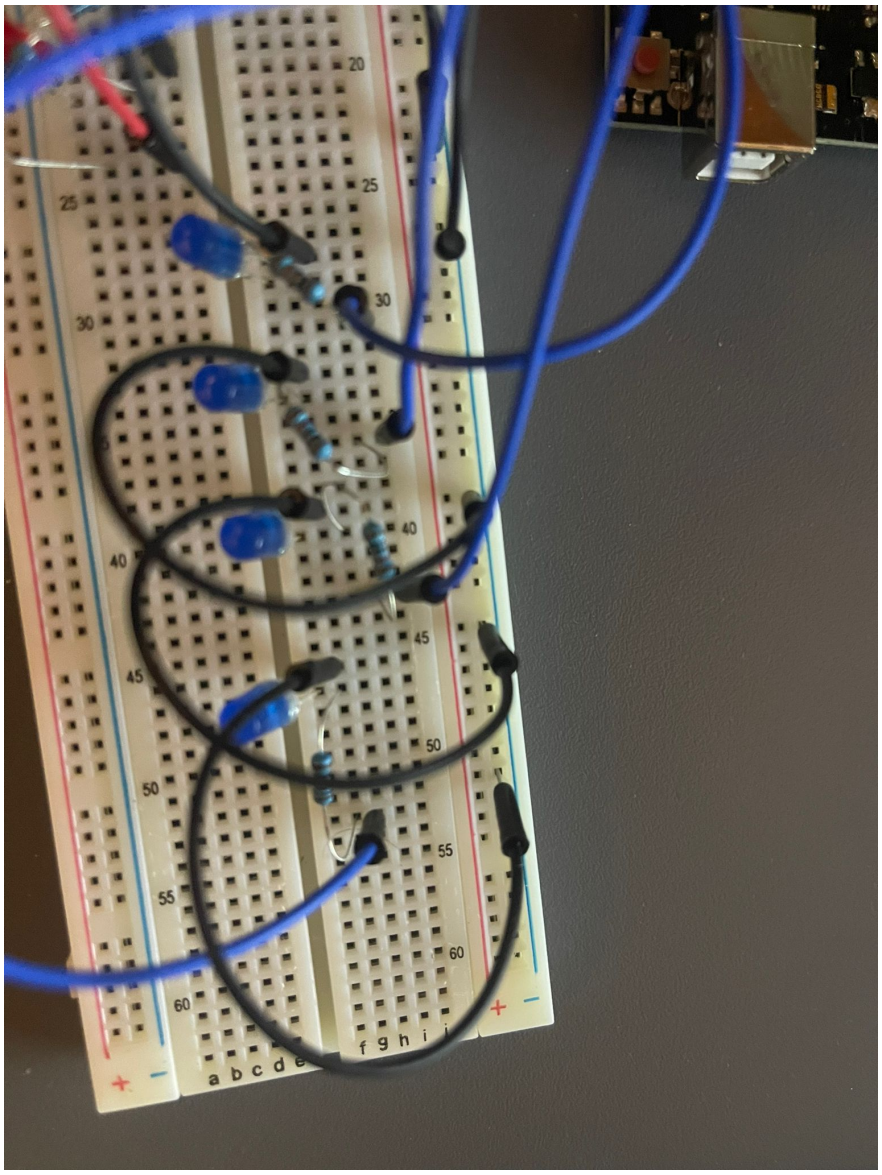


Figure 2: LED outputs for test cases.

3.5 Discussion

This lab demonstrated how bitwise operations can be used to monitor multiple subsystems in an embedded environment. The implementation successfully mapped status bits to LEDs and evaluated logical conditions. Challenges included debugging bitwise logic and ensuring correct hardware wiring. The pseudocode helped clarify the program structure before implementation.

4 References

References

[1] Elliot Williams, *Make: AVR Programming*, O'Reilly Media.

[2] HKR Course Materials: DT124A Embedded Systems Labs.

[3] Microchip Studio Documentation.